In [2]:
```
import pandas as pd;
```

In [4]:
```
transactions = pd.read_csv("transactions.csv");
```

In [6]:
```
customers = pd.read_csv("customer.csv");
```

In [7]:
```
transactions.head()
```

Out[7]:

	Transaction_ID	Customer_ID	Transaction_DateTime	Amount_GBP	Debit_Credit	Tran
0	TX1000000	106875	2025-07-05 20:16:00	8.88	Debit	C
1	TX1000001	106413	2025-02-25 01:29:00	26.03	Debit	C
2	TX1000002	100870	2025-04-24 12:43:00	12.53	Debit	C
3	TX1000003	101974	2025-06-21 04:45:00	23.49	Debit	Cas
4	TX1000004	102268	2025-07-12 21:30:00	61.66	Debit	Cas

In [9]:
```
transactions.head(10)
```

Out[9]:

	Transaction_ID	Customer_ID	Transaction_DateTime	Amount_GBP	Debit_Credit	Tran
0	TX1000000	106875	2025-07-05 20:16:00	8.88	Debit	C
1	TX1000001	106413	2025-02-25 01:29:00	26.03	Debit	C
2	TX1000002	100870	2025-04-24 12:43:00	12.53	Debit	C
3	TX1000003	101974	2025-06-21 04:45:00	23.49	Debit	Cas
4	TX1000004	102268	2025-07-12 21:30:00	61.66	Debit	Cas
5	TX1000005	101631	2025-04-28 08:17:00	79.54	Debit	
6	TX1000006	101243	2025-09-24 14:38:00	49.69	Debit	Cas
7	TX1000007	105554	2025-12-06 06:16:00	3.28	Debit	C
8	TX1000008	106826	2025-06-20 08:59:00	143.00	Debit	
9	TX1000009	105540	2025-08-19 22:17:00	12.75	Debit	C

In [ ]:
```
# the last 10 bottom transactions
```

In [10]:
```
transactions.tail(10)
```

Out[10]:

	Transaction_ID	Customer_ID	Transaction_DateTime	Amount_GBP	Debit_Credit
99990	TX1099990	106094	2025-07-21 10:22:00	42.24	Debit
99991	TX1099991	102171	2025-01-15 03:14:00	43.95	Credit
99992	TX1099992	103306	2025-05-01 21:08:00	15.51	Debit
99993	TX1099993	100021	2025-05-14 23:09:00	8.81	Debit
99994	TX1099994	103182	2025-08-06 18:44:00	46.47	Debit
99995	TX1099995	104693	2025-09-20 13:18:00	87.22	Debit
99996	TX1099996	106523	2025-02-23 17:02:00	32.58	Debit
99997	TX1099997	104512	2025-04-29 12:42:00	48.23	Debit
99998	TX1099998	103797	2025-05-27 17:37:00	73.03	Credit
99999	TX1099999	101340	2025-03-29 23:18:00	117.77	Debit

In [11]:
```
transactions.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 16 columns):
 #   Column                Non-Null Count   Dtype  
---  ------                --------------   -----  
 0   Transaction_ID        100000 non-null  str    
 1   Customer_ID           100000 non-null  int64  
 2   Transaction_DateTime  100000 non-null  str    
 3   Amount_GBP            100000 non-null  float64
 4   Debit_Credit          100000 non-null  str    
 5   Transaction_Type      100000 non-null  str    
 6   Channel               100000 non-null  str    
 7   Merchant_Category     100000 non-null  str    
 8   Merchant_City         100000 non-null  str    
 9   Customer_Region       100000 non-null  str    
 10  Account_Type          100000 non-null  str    
 11  Status                100000 non-null  str    
 12  Balance_Before_GBP    100000 non-null  float64
 13  Balance_After_GBP     100000 non-null  float64
 14  Fraud_Flag            100000 non-null  bool   
 15  Fraud_Reason          293 non-null     str    
dtypes: bool(1), float64(3), int64(1), str(11)
memory usage: 20.9 MB
```

In [12]:
```
transactions.describe()
```

Out[12]:

	Customer_ID	Amount_GBP	Balance_Before_GBP	Balance_After_GBP
count	100000.000000	100000.000000	100000.000000	100000.000000
mean	103997.455840	87.694677	2221.207003	2177.629669
std	2311.281887	176.819359	2936.378393	2942.414742
min	100001.000000	1.000000	14.140000	-5097.670000
25%	101994.750000	21.920000	683.180000	645.830000
50%	103997.000000	41.960000	1339.595000	1304.505000
75%	105993.000000	82.492500	2634.272500	2601.922500
max	108000.000000	5774.260000	128065.370000	128055.580000

In [13]:
```
customers.head()
```

Out[13]:

	Customer_ID	Region	Age_Band	Account_Type	Account_Tenure_Years
0	100001	South East	45-54	Savings	15.5
1	100002	London	65+	Current	10.8
2	100003	East of England	35-44	Current	3.8
3	100004	South East	18-24	Current	3.6
4	100005	West Midlands	55-64	Current	6.1

In [ ]:
```
# the first top 20 customers
```

In [15]:
```
customers.head(20)
```

Out[15]:

	Customer_ID	Region	Age_Band	Account_Type	Account_Tenure_Years
0	100001	South East	45-54	Savings	15.5
1	100002	London	65+	Current	10.8
2	100003	East of England	35-44	Current	3.8
3	100004	South East	18-24	Current	3.6
4	100005	West Midlands	55-64	Current	6.1
5	100006	Wales	55-64	Premier	6.1
6	100007	South East	45-54	Savings	2.8
7	100008	South West	55-64	Current	16.7
8	100009	West Midlands	55-64	Current	5.7
9	100010	London	45-54	Savings	16.1
10	100011	Yorkshire and Humber	65+	Current	19.5
11	100012	East of England	55-64	Current	6.7
12	100013	London	45-54	Savings	19.4
13	100014	South West	35-44	Current	3.9
14	100015	London	18-24	Current	13.0
15	100016	North West	25-34	Current	19.8
16	100017	London	55-64	Current	16.2
17	100018	East Midlands	55-64	Savings	4.7
18	100019	South West	35-44	Savings	5.2
19	100020	London	45-54	Premier	5.3

In [ ]:
```
the last bottom 20 customers
```

In [16]:
```
customers.tail(20)
```

Out[16]:

	Customer_ID	Region	Age_Band	Account_Type	Account_Tenure_Years
7980	107981	Yorkshire and Humber	18-24	Current	15.4
7981	107982	East of England	25-34	Current	15.0
7982	107983	East of England	55-64	Current	10.5
7983	107984	Wales	25-34	Premier	14.9
7984	107985	South East	25-34	Savings	6.3
7985	107986	West Midlands	25-34	Current	4.4
7986	107987	North East	55-64	Savings	2.6
7987	107988	North East	45-54	Current	5.9
7988	107989	South East	18-24	Current	13.1
7989	107990	East of England	35-44	Savings	5.2
7990	107991	London	25-34	Savings	17.7
7991	107992	North East	65+	Savings	5.1
7992	107993	West Midlands	25-34	Savings	13.7
7993	107994	East of England	45-54	Current	10.6
7994	107995	East Midlands	35-44	Premier	4.3
7995	107996	Yorkshire and Humber	45-54	Current	11.2
7996	107997	West Midlands	65+	Current	19.1
7997	107998	London	35-44	Savings	18.9
7998	107999	North West	55-64	Current	5.9
7999	108000	North East	35-44	Current	10.8

In [17]:
```
customers.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 5 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   Customer_ID          8000 non-null   int64  
 1   Region               8000 non-null   str    
 2   Age_Band             8000 non-null   str    
 3   Account_Type         8000 non-null   str    
 4   Account_Tenure_Years  8000 non-null   float64
dtypes: float64(1), int64(1), str(3)
memory usage: 490.0 KB
```

In [18]:
```
customers.describe()
```

Out[18]:

	Customer_ID	Account_Tenure_Years
count	8000.00000	8000.000000
mean	104000.50000	10.084113
std	2309.54541	5.726084
min	100001.00000	0.100000
25%	102000.75000	5.100000
50%	104000.50000	10.100000
75%	106000.25000	15.000000
max	108000.00000	20.000000

In [19]:
```
customers.describe(include="all")
```

Out[19]:

	Customer_ID	Region	Age_Band	Account_Type	Account_Tenure_Years
count	8000.00000	8000	8000	8000	8000.000000
unique	NaN	10	6	4	NaN
top	NaN	London	35-44	Current	NaN
freq	NaN	1768	1698	4884	NaN
mean	104000.50000	NaN	NaN	NaN	10.084113
std	2309.54541	NaN	NaN	NaN	5.726084
min	100001.00000	NaN	NaN	NaN	0.100000
25%	102000.75000	NaN	NaN	NaN	5.100000
50%	104000.50000	NaN	NaN	NaN	10.100000
75%	106000.25000	NaN	NaN	NaN	15.000000
max	108000.00000	NaN	NaN	NaN	20.000000

In [21]:
```
customers.isnull().sum()
```

Out[21]:
```
Customer_ID             0
Region                  0
Age_Band                0
Account_Type            0
Account_Tenure_Years    0
dtype: int64
```

In [22]:
```
transactions.isnull().sum()
```

Out[22]:
```
Transaction_ID              0
Customer_ID                 0
Transaction_DateTime        0
Amount_GBP                  0
Debit_Credit                0
Transaction_Type            0
Channel                     0
Merchant_Category           0
Merchant_City               0
Customer_Region             0
Account_Type                0
Status                      0
Balance_Before_GBP          0
Balance_After_GBP           0
Fraud_Flag                  0
Fraud_Reason            99707
dtype: int64
```

In [24]:
```
transactions.columns = transactions.columns.str.lower()
```

In [25]:
```
customers.columns = customers.columns.str.lower()
```

In [26]:
```
transactions.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 16 columns):
 #   Column                Non-Null Count   Dtype  
---  ------                --------------   -----  
 0   transaction_id        100000 non-null  str    
 1   customer_id           100000 non-null  int64  
 2   transaction_datetime  100000 non-null  str    
 3   amount_gbp            100000 non-null  float64
 4   debit_credit          100000 non-null  str    
 5   transaction_type      100000 non-null  str    
 6   channel               100000 non-null  str    
 7   merchant_category     100000 non-null  str    
 8   merchant_city         100000 non-null  str    
 9   customer_region       100000 non-null  str    
 10  account_type          100000 non-null  str    
 11  status                100000 non-null  str    
 12  balance_before_gbp    100000 non-null  float64
 13  balance_after_gbp     100000 non-null  float64
 14  fraud_flag            100000 non-null  bool   
 15  fraud_reason          293 non-null     str    
dtypes: bool(1), float64(3), int64(1), str(11)
memory usage: 20.9 MB
```

In [27]:
```
customers.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   customer_id           8000 non-null   int64
 1   region                8000 non-null   str
 2   age_band              8000 non-null   str
 3   account_type          8000 non-null   str
 4   account_tenure_years  8000 non-null   float64
dtypes: float64(1), int64(1), str(3)
memory usage: 490.0 KB
```

In [30]:
```
customers["account_tenure_years"] = customers["account_tenure_years"].round()
```

In [31]:
```
customers.head()
```

Out[31]:

	customer_id	region	age_band	account_type	account_tenure_years
0	100001	South East	45-54	Savings	16.0
1	100002	London	65+	Current	11.0
2	100003	East of England	35-44	Current	4.0
3	100004	South East	18-24	Current	4.0
4	100005	West Midlands	55-64	Current	6.0

In [37]:
```
customers["account_tenure_years"] = customers["account_tenure_years"].astype(int
```

In [33]:
```
customers.head()
```

Out[33]:

	customer_id	region	age_band	account_type	account_tenure_years
0	100001	South East	45-54	Savings	16.0
1	100002	London	65+	Current	11.0
2	100003	East of England	35-44	Current	4.0
3	100004	South East	18-24	Current	4.0
4	100005	West Midlands	55-64	Current	6.0

In [38]:
```
customers.head()
```

Out[38]:

	customer_id	region	age_band	account_type	account_tenure_years
0	100001	South East	45-54	Savings	16
1	100002	London	65+	Current	11
2	100003	East of England	35-44	Current	4
3	100004	South East	18-24	Current	4
4	100005	West Midlands	55-64	Current	6

In [39]:
```
transactions.head()
```

Out[39]:

	transaction_id	customer_id	transaction_datetime	amount_gbp	debit_credit	transact
0	TX1000000	106875	2025-07-05 20:16:00	8.88	Debit	Card
1	TX1000001	106413	2025-02-25 01:29:00	26.03	Debit	Card
2	TX1000002	100870	2025-04-24 12:43:00	12.53	Debit	Card
3	TX1000003	101974	2025-06-21 04:45:00	23.49	Debit	Cash W
4	TX1000004	102268	2025-07-12 21:30:00	61.66	Debit	Cash W

In [41]:
```
from sqlalchemy import create_engine
import urllib

connection_string = urllib.parse.quote_plus(
    'DRIVER={ODBC Driver 17 for SQL Server};'
    'SERVER=AyoDAnalyst\\SQLEXPRESS01;'
    'DATABASE=finance;'
    'Trusted_Connection=yes;'
)

engine = create_engine(
    f'mssql+pyodbc:///?odbc_connect={connection_string}'
)
```

In [43]:
```
transactions.to_sql(
    'transactions',
    engine,
    if_exists='replace',
    index=False
)
```

Out[43]: 47

In [44]:
```
customers.to_sql(
    'customers',
    engine,
    if_exists='replace',
    index=False
)
```

Out[44]: 39

In [ ]:
```

```